

SPA con React

3. Uso de API REST



Universidad
Rey Juan Carlos

Micael Gallego

Correo: micael.gallego@urjc.es

Twitter: [@micael_gallego](https://twitter.com/micael_gallego)



- React permite que las peticiones a una **API REST** se realicen con el cliente nativo del navegador **fetch(...)** usando **async/await**
- Se verá cómo realizar las peticiones para que la interfaz se actualice cuando llegue la respuesta

- Ejemplo de **buscador libros** en OpenLibrary



<https://openlibrary.org/search.json?title=JavaScript>

Cliente REST

```
import { useRef, useState } from "react";

export default function Home() {

  const [titles, setTitles] = useState<string[]>([]);
  const [error, setError] = useState<string | null>(null);
  const [isPending, setIsPending] = useState(false);

  const titleRef = useRef<HTMLInputElement>(null);

  async function handleSubmit(event: React.SubmitEvent) {...};

  return (
    <div>
      <h1>Open Library</h1>

      <form onSubmit={handleSubmit}>
        <input type="text" ref={titleRef} disabled={isPending} />
        <button type="submit" disabled={isPending}>Buscar</button>
      </form>

      {isPending && <p>Cargando...</p>}

      {error && <p style={{ color: "red" }}>{error}</p>}

      {!isPending && titles.length === 0 && !error && <p>No se encontraron resultados</p>}

      {!isPending && titles.map((title: string, index: number) => (
        <p key={index}>{title}</p>
      ))}
    </div>
  );
}
```

Cliente REST

```
import { useRef, useState } from "react";
```

```
export default function Home() {
```

```
  const [titles, setTitles] = useState<string[]>([]);  
  const [error, setError] = useState<string | null>(null);  
  const [isPending, setIsPending] = useState(false);
```

Estado para el resultado
de la búsqueda (títulos)

Estado para controlar la
llamada REST

```
  const titleRef = useRef<HTMLInputElement>(null);
```

```
  async function handleSubmit(event: React.SubmitEvent) {...};
```

```
  return (
```

```
    <div>
```

```
      <h1>Open Library</h1>
```

```
      <form onSubmit={handleSubmit}>
```

```
        <input type="text" ref={titleRef} disabled={isPending} />
```

```
        <button type="submit" disabled={isPending}>Buscar</button>
```

```
      </form>
```

Mientras carga se adapta
la interfaz (isPending)

```
      {isPending && <p>Cargando...</p>}
```

```
      {error && <p style={{ color: "red" }}>{error}</p>}
```

Si hay error

```
      {!isPending && titles.length === 0 && !error && <p>No se encontraron resultados</p>}
```

```
      {!isPending && titles.map((title: string, index: number) => (  
        <p key={index}>{title}</p>  
      ))}
```

Cuando ha cargado se
muestra el resultado

```
    </div>
```

```
  );
```

```
}
```

Cliente REST

home.tsx

```
...
async function handleSubmit(event: React.SubmitEvent) {

  event.preventDefault();

  setTitles([]);
  setError(null);
  setIsPending(true);

  const searchTerm = titleRef.current?.value ?? "";

  const url = `https://openlibrary.org/search.json?title=${encodeURIComponent(searchTerm)}`;

  try {
    const response = await fetch(url);
    const data = await response.json();

    if (data.docs) {
      const bookTitles = data.docs.map((item: any) => item.title);
      setTitles(bookTitles);
    }
  } catch (error) {
    console.error(error);
    setError("Error al buscar libros");
  } finally {
    setIsPending(false);
  }

  event.target.reset();
};
```

Ciente REST

home.tsx

```
...
async function handleSubmit(event: React.SubmitEvent) {
  event.preventDefault();

  setTitles([]);
  setError(null);
  setIsPending(true);

  const searchTerm = titleRef.current?.value ?? "";

  const url = `https://openlibrary.org/search.json?title=${encodeURIComponent(searchTerm)}`;

  try {
    const response = await fetch(url);
    const data = await response.json();

    if (data.docs) {
      const bookTitles = data.docs.map((item: any) => item.title);
      setTitles(bookTitles);
    }
  } catch (error) {
    console.error(error);
    setError("Error al buscar libros");
  } finally {
    setIsPending(false);
  }

  event.target.reset();
};
```

Evita el comportamiento nativo

Inicializa el estado

Petición con fetch

Los resultados se guardan en el estado

Se reinicia el formulario

useState

- Este patrón es tan común que el *hook* **useState** lo simplifica
- Se gestionan la carga (isLoading), el error y el comportamiento por defecto del formulario de forma automática

useActionState

```
import { useActionState } from "react";
...
export default function Home() {

  const [{ titles, error }, formAction, isPending] = useActionState(
    searchAction,
    { titles: new Array<string>(), error: null }
  );

  return (
    <div>
      <h1>Open Library</h1>

      <form action={formAction}>
        <input type="text" name="title" disabled={isPending} />
        <button type="submit" disabled={isPending}>Buscar</button>
      </form>

      {isPending && <p>Cargando...</p>}

      {error && <p style={{ color: "red" }}>{error}</p>}

      {!isPending && titles.length === 0 && !error && <p>No se encontraron resultados</p>}

      {!isPending && titles.map((title: string, index: number) => (
        <p key={index}>{title}</p>
      ))}
    </div>
  );
}
```

useActionState

```
import { useActionState } from "react";
...
export default function Home() {

  const [{ titles, error }, formAction, isPending] = useActionState(
    searchAction,
    { titles: new Array<string>(), error: null }
  );

  return (
    <div>
      <h1>Open Library</h1>

      <form action={formAction}>
        <input type="text" name="title" disabled={isPending} />
        <button type="submit" disabled={isPending}>Buscar</button>
      </form>

      {isPending && <p>Cargando...</p>}

      {error && <p style={{ color: "red" }}>{error}</p>}

      {!isPending && titles.length === 0 && !error && <p>No se encontraron resultados</p>}

      {!isPending && titles.map((title: string, index: number) => (
        <p key={index}>{title}</p>
      ))}
    </div>
  );
}
```

Gestiona isPending, el estado inicial y la función de carga

La plantilla no cambia

useActionState

exam27_rest_action

```
async function searchAction(
  prevState: { titles: string[], error: string | null }, formData: FormData) {

  const searchTerm = (formData.get("title") as string) ?? "";

  const url = `https://openlibrary.org/search.json?title=${encodeURIComponent(searchTerm)}`;

  try {
    const response = await fetch(url);
    const data = await response.json();

    if (data.docs) {
      const bookTitles = data.docs.map((item: any) => item.title);
      return { titles: bookTitles, error: null };
    }

    return { titles: [], error: null };
  } catch (error) {

    console.error(error);
    return { titles: [], error: "Error al buscar libros" };
  }
}
```

useActionState

exam27_rest_action

El preventDefault y reset
son automáticos

Se accede a los datos del
form con FormData

```
async function searchAction(  
  prevState: { titles: string[], error: string | null }, formData: FormData) {  
  
  const searchTerm = (formData.get("title") as string) ?? "";  
  
  const url = `https://openlibrary.org/search.json?title=${encodeURIComponent(searchTerm)}`;  
  
  try {  
    const response = await fetch(url);  
    const data = await response.json();  
  
    if (data.docs) {  
      const bookTitles = data.docs.map((item: any) => item.title);  
      return { titles: bookTitles, error: null };  
    }  
  
    return { titles: [], error: null };  
  } catch (error) {  
  
    console.error(error);  
    return { titles: [], error: "Error al buscar libros" };  
  }  
}
```

El nuevo estado se
devuelve

API REST en servicios

- Para que el código sea más **mantenible** es recomendable separar las responsabilidades de los módulos de la aplicación
- Se implementa un servicio (**titles-service.ts**) encargado de realizar las peticiones de la API REST y se usa ese servicio desde el componente
- Un servicio es un **módulo TypeScript** que no depende de React

- Servicio API REST

titles-service.ts

```
export async function searchTitles(searchTerm: string): Promise<string[]> {  
  
  const url = `https://openlibrary.org/search.json?title=${encodeURIComponent(searchTerm)}`;  
  
  const response = await fetch(url);  
  const data = await response.json();  
  
  if (data.docs) {  
    return data.docs.map((item: any) => item.title);  
  }  
  
  return [];  
}
```

- Uso del servicio de la API REST

home.tsx

```
...
async function searchAction(
  prevState: { titles: string[], error: string | null }, formData: FormData) {

  const searchTerm = formData.get("title") as string;

  try {
    const results = await searchTitles(searchTerm);
    return { titles: results, error: null };
  } catch (error) {
    console.error(error);
    return { titles: [], error: "Error al buscar libros" };
  }
}
...
```

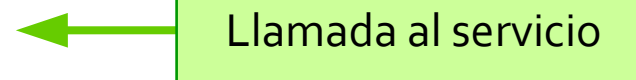
- Uso del servicio de la API REST

home.tsx

```
...
async function searchAction(
  prevState: { titles: string[], error: string | null }, formData: FormData) {

  const searchTerm = formData.get("title") as string;

  try {
    const results = await searchTitles(searchTerm);
    return { titles: results, error: null };
  } catch (error) {
    console.error(error);
    return { titles: [], error: "Error al buscar libros" };
  }
}
...
```



A green box with the text "Llamada al servicio" (Call to the service) has a green arrow pointing to the line `const results = await searchTitles(searchTerm);` in the code block.

API REST en Store

- La información de la API REST también se puede gestionar en un **store** (para compartirla entre componentes)
- El componente usa **useActionState** para gestión de formulario, pero el estado lo lee del **store**

```
import { create } from "zustand";

interface TitlesState {
  titles: string[];
  error: string | null;
  searchTitles: (searchTerm: string) => Promise<void>;
}

async function fetchTitles(searchTerm: string): Promise<string[]> {
  const url = `https://openlibrary.org/search.json?title=${encodeURIComponent(searchTerm)}`;
  const response = await fetch(url);
  const data = await response.json();

  if (data.docs) {
    return data.docs.map((item: any) => item.title);
  }
  return [];
}

export const useTitlesStore = create<TitlesState>((set) => ({
  titles: [],
  error: null,

  searchTitles: async (searchTerm: string) => {
    set({ titles: null, error: null });

    try {
      const bookTitles = await fetchTitles(searchTerm);
      set({ titles: bookTitles });
    } catch (error) {
      console.error(error);
      set({ error: "Error al buscar libros" });
    }
  },
}));
```

```
import { create } from "zustand";

interface TitlesState {
  titles: string[];
  error: string | null;
  searchTitles: (searchTerm: string) => Promise<void>;
}

async function fetchTitles(searchTerm: string): Promise<string[]> {
  const url = `https://openlibrary.org/search.json?title=${encodeURIComponent(searchTerm)}`;
  const response = await fetch(url);
  const data = await response.json();

  if (data.docs) {
    return data.docs.map((item: any) => item.title);
  }
  return [];
}

export const useTitlesStore = create<TitlesState>((set) => ({
  titles: [],
  error: null,

  searchTitles: async (searchTerm: string) => {
    set({ titles: null, error: null });

    try {
      const bookTitles = await fetchTitles(searchTerm);
      set({ titles: bookTitles });
    } catch (error) {
      console.error(error);
      set({ error: "Error al buscar libros" });
    }
  },
}));
```

Operación de carga

Estado

El estado se actualiza con la respuesta de fetch(...)

```
import { useActionState } from "react";
import { useTitlesStore } from "~/stores/titles-store";

export default function Home() {

  const { error, titles, searchTitles } = useTitlesStore();

  async function searchAction(prevState: any, formData: FormData) {
    const searchTerm = formData.get("title") as string;
    await searchTitles(searchTerm);
  }

  const [_, formAction, isPending] = useActionState(searchAction, null);

  return (
    <div>
      <h1>Open Library</h1>

      <form action={formAction}>
        <input type="text" name="title" disabled={isPending} />
        <button type="submit" disabled={isPending}>Buscar</button>
      </form>

      {isPending && <p>Cargando...</p>}
      {error && <p style={{ color: "red" }}>{error}</p>}
      {!isPending && titles.length === 0 && !error && <p>No se encontraron resultados</p>}
      {!isPending && titles.map((title, index) => (
        <p key={index}>{title}</p>
      ))}
    </div>
  );
}
```

```
import { useActionState } from "react";
import { useTitlesStore } from "~/stores/titles-store";

export default function Home() {

  const { error, titles, searchTitles } = useTitlesStore();

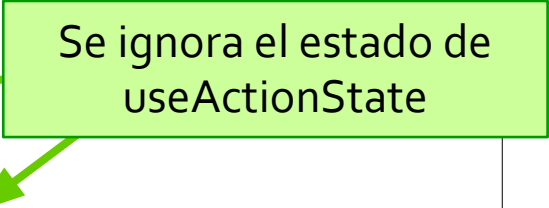
  async function searchAction(prevState: any, formData: FormData) {
    const searchTerm = formData.get("title") as string;
    await searchTitles(searchTerm);
  }

  const [_, formAction, isPending] = useActionState(searchAction, null);

  return (
    <div>
      <h1>Open Library</h1>

      <form action={formAction}>
        <input type="text" name="title" disabled={isPending} />
        <button type="submit" disabled={isPending}>Buscar</button>
      </form>

      {isPending && <p>Cargando...</p>}
      {error && <p style={{ color: "red" }}>{error}</p>}
      {!isPending && titles.length === 0 && !error && <p>No se encontraron resultados</p>}
      {!isPending && titles.map((title, index) => (
        <p key={index}>{title}</p>
      ))}
    </div>
  );
}
```



Se ignora el estado de useActionState

Carga automática

- Se pueden cargar los datos de forma **automática**
- La petición a la API REST se realiza al **renderizar** el componente por primera vez
- Como no se inicia con un formulario, no se recomienda usar **useActionState**
- Se usa el *hook* **useEffect**

<https://react.dev/reference/react/useEffect>

Carga automática

```
import { useEffect, useState } from "react";

export default function Home() {

  const [titles, setTitles] = useState<string[]>([]);
  const [error, setError] = useState<string | null>(null);
  const [isPending, setIsPending] = useState(true);

  async function fetchBooks() { ... }

  useEffect(() => { fetchBooks() }, []);

  return (
    <div>...</div>
  );
}
```

Carga automática

```
import { useEffect, useState } from "react";

export default function Home() {

  const [titles, setTitles] = useState<string[]>([]);
  const [error, setError] = useState<string | null>(null);
  const [isPending, setIsPending] = useState(true);

  async function fetchBooks() { ... }

  useEffect(() => { fetchBooks() }, []);

  return (
    <div>...</div>
  );
}
```

La gestión del estado es igual

La llamada a la API REST es igual

Se ejecuta al cargar el componente por primera vez

Carga automática

- `useEffect(setup, dependencies?)`
 - `setup`: acción que se ejecuta
 - `dependencies`:
 - Lista de valores que si cambian ejecutarán de nuevo la acción
 - Si la lista está vacía, la acción sólo se ejecuta cuando carga el componente por primera vez

<https://react.dev/reference/react/useEffect>

Ejercicio 6

- Añade al ejercicio 5 de **gestión de elementos** la gestión en el servidor (mediante API REST)
- Modifica el **store** para que las operaciones sobre los datos se realicen en una API REST mediante llamadas a la API REST
- Actualiza los **items** cada vez que se realice una petición al servidor

Ejercicio 6

- Se proporciona un servicio web que exporta una API REST

Java

```
$ cd exer6_items_backend/java  
$ mvn spring-boot:run
```

Node

```
$ cd exer6_items_backend/node  
$ npm start
```



POSTMAN

```
Items.postman_collection.json
```

Ejercicio 6

• API REST Items

- Creación de items
 - Method: POST
 - URL: `http://127.0.0.1:8080/api/items/`
 - Headers: Content-Type: `application/json`
 - Body:

```
{ "description" : "Leche", "checked": false }
```

- Result:

```
{ "id": 1, "description" : "Leche", "checked": false }
```

- Status code: `201 (Created)`

Ejercicio 6

- **API REST Items**

- Consulta de items
 - Method: GET
 - URL: `http://127.0.0.1:8080/api/items/`
 - Result:

```
[  
  { "id": 1, "description": "Leche", "checked": false },  
  { "id": 2, "description": "Pan", "checked": true }  
]
```

- Status code: 200 (OK)

Ejercicio 6

- **API REST Items**

- Modificación de items

- Method: PUT

- URL: `http://127.0.0.1:8080/api/items/1`

- Headers: Content-Type: application/json

- Body:

```
{ "id": 1, "description" : "Leche", "checked": true }
```

- Result:

```
{ "id": 1, "description" : "Leche", "checked": true }
```

- Status code: 200 (OK) / 404 (Not Found)

Ejercicio 6

- **API REST Items**

- Modificación de items
 - Method: DELETE
 - URL: `http://127.0.0.1:8080/api/items/1`
 - Result:

```
{ "id": 1, "description" : "Leche", "checked": true }
```

- Status code: 200 (OK) / 404 (Not Found)

Web y API en diferente dominio/puerto

- Por **seguridad**, si una web se carga en un dominio/puerto, por defecto **no puede realizar** peticiones a una API REST en otro dominio y puerto
- Una aplicación React cargada desde **http://localhost:5173** por defecto no puede acceder a una API REST en **http://localhost:8080**
- Se puede configurar el servidor de la API REST para que admita peticiones de otros dominios/puertos (**CORS, *Cross-origin Resource Sharing***)

El backend del ejercicio 6 tiene configurado CORS

Web y API en diferente dominio/puerto

- Si se la web se va a desplegar en el mismo dominio/puerto que la API REST es mejor **no configurar CORS** (más seguro y eficiente)
- Es mejor usar **rutas relativas** en React al hacer peticiones REST
 - **/api** en vez de **http://localhost:8080/api**
- En desarrollo hay que configurar un **proxy** en React Router para que funcionen las rutas relativas
 - El proxy redirige las peticiones que le llegan a la API REST (**/api**) al servidor real en **http://localhost:8080/api**

Web y API en diferente dominio/puerto

- Proxy en React Router

vite.config.ts

```
export default defineConfig({  
  plugins: [tailwindcss(), reactRouter(), tsconfigPaths()],  
});
```



```
export default defineConfig({  
  plugins: [tailwindcss(), reactRouter(), tsconfigPaths()],  
  server: {  
    proxy: {  
      '/api': {  
        target: 'http://localhost:8080/api',  
        changeOrigin: true,  
        rewrite: (path) => path.replace(/^\/api/, '')  
      }  
    }  
  }  
});
```

Ejercicio 7

- Adapta el Ejercicio 6 para que las llamadas a la API REST sean relativas

Uso de API REST

- En React se usan las API REST con el **fetch nativo** del navegador
- Se puede llamar a una API REST en el evento de **botón pulsado**
- Se puede usar una API REST al carga la página (*hook* **useEffect**)
- Para procesar formularios se usa el *hook* **useActionState**
- Si el resultado de la API se tiene que compartir entre varios componentes, las llamadas a la API REST se pueden gestionar en un **store**